ASSIGMENT-16.1

Name: Sameera Anjum

HT. NO:2303A52466

BATCH:40

Task 1 – Student Information System Schema

Task:

Design a database schema for a Student Information System and generate queries using AI.

Instructions:

- Tables: Students, Courses, Enrollments.
- Define primary keys, foreign keys, and relationships.
- Generate queries:
    - Insert a new student record.
    - Fetch all courses enrolled by a student.
    - Count number of students in each course.

Expected Output:

- SQL CREATE TABLE statements and queries for student–course relationships.

```sql
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    department VARCHAR(50)
);
CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100),
    credits INT
);
CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    enrollment_date DATE,
```

```sql
    enrollment_date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
INSERT INTO Students (student_id, name, email, department)
VALUES (101, 'Rahul Sharma', 'rahul@example.com', 'Computer Science');
SELECT s.name, c.course_name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
WHERE s.student_id = 101;
SELECT c.course_name, COUNT(e.student_id) AS total_students
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
```

OUTPUT:

Students

student_id	name	email	department
101	Rahul Sharma	rahul@example.com	Computer Science

PROMT:

Design a normalized SQL schema for a Student Information System with tables Students, Courses, and Enrollments. Include primary keys and foreign keys. Generate SQL queries to insert a student, fetch all courses of a student, and count students in each course.

Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

• Tables: Doctors, Patients, Appointments.

• Use AI to define constraints (unique IDs, valid dates).

• Generate queries:

  o List all appointments for a specific doctor.

  o Retrieve patient history by patient ID.

  o Count total patients treated by each doctor.

Expected Output:

• Normalized schema and SQL queries with joins.

```sql
CREATE TABLE Doctors (
    doctor_id INT PRIMARY KEY,
    name VARCHAR(100),
    specialization VARCHAR(100)
);

CREATE TABLE Patients (
    patient_id INT PRIMARY KEY,
    name VARCHAR(100),
    age INT,
    gender VARCHAR(10)
);

CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY,
    doctor_id INT,
    patient_id INT,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
INSERT INTO Doctors (doctor_id, name, specialization) VALUES
(1, 'Dr. Ravi Kumar', 'Cardiologist'),
(2, 'Dr. Priya Sharma', 'Dermatologist'),
(3, 'Dr. Arjun Mehta', 'Orthopedic'),
(4, 'Dr. Sneha Iyer', 'Pediatrician');
INSERT INTO Patients (patient_id, name, age, gender) VALUES
(101, 'Amit Verma', 30, 'Male'),
(102, 'Neha Gupta', 25, 'Female'),
(103, 'Rahul Das', 40, 'Male'),
(104, 'Pooja Nair', 35, 'Female');
INSERT INTO Appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES
```

OUTPUT:

• Tables: Doctors, Patients, Appointments.

```
SELECT d.name AS doctor_name, p.name AS patient_name, a.appointment_date
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
WHERE d.doctor id = 1;
```

• Use AI to define constraints (unique IDs, valid dates).

```
SELECT p.name, d.name AS doctor, a.appointment_date
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
WHERE p.patient_id = 1;
```

• Generate queries:

  o List all appointments for a specific doctor.

  o Retrieve patient history by patient ID.

  o Count total patients treated by each doctor.

```
SELECT d.name, COUNT(a.patient_id) AS total_patients
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.name;
```

PROMT:

Create a SQL schema for a Hospital Management System with Doctors, Patients, and Appointments. Add constraints like primary keys and valid dates. Generate queries to list appointments by doctor, retrieve patient history, and count patients per doctor.

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

• Tables: Books, Members, Loans.

• Use AI to suggest indexing strategy for faster queries.

• Generate queries:

  o Retrieve all books currently issued.

  o Find overdue books (loan date > 30 days).

  o Count number of books loaned by each member.

Expected Output:

• Schema + SQL queries demonstrating joins and conditions.

```
CREATE TABLE Members (
    member_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);
CREATE TABLE Books (
    book_id INT PRIMARY KEY,
    title VARCHAR(200),
    author VARCHAR(100),
    available_copies INT
);
CREATE TABLE Loans (
    loan_id INT PRIMARY KEY,
    book_id INT,
    member_id INT,
    loan_date DATE,
```

```
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
CREATE INDEX idx_book_id ON Loans(book_id);
CREATE INDEX idx_member_id ON Loans(member_id);
INSERT INTO Members (member_id, name, email) VALUES
(1, 'Anil Kumar', 'anil@gmail.com'),
(2, 'Divya Sharma', 'divya@gmail.com'),
(3, 'Karthik R', 'karthik@gmail.com');
INSERT INTO Books (book_id, title, author, available_copies) VALUES
(101, 'Database Systems', 'Navathe', 5),
(102, 'Operating Systems', 'Galvin', 3),
(103, 'Python Programming', 'Guido', 4);
INSERT INTO Loans (loan_id, book_id, member_id, loan_date, return_date) VALUES
(1, 101, 1, '2026-02-10', NULL),
(2, 102, 2, '2026-01-15', NULL),
(3, 103, 3, '2026-03-01', '2026-03-10');
```

OUTPUT:

• Tables: Books, Members, Loans.

```
SELECT b.title, m.name
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
```

• Use AI to suggest indexing strategy for faster queries.

```
SELECT b.title, m.name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE CURRENT_DATE - l.loan_date > 30
AND l.return_date IS NULL;
```

• Generate queries:

  o Retrieve all books currently issued.

  o Find overdue books (loan date > 30 days).

o Count number of books loaned by each member.

```
SELECT m.name, COUNT(l.book_id) AS total_books
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.name;
```

OUTPUT:


```
SELECT m.name, COUNT(l.book_id) AS total_books
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.name;
```


PROMT:

Design a normalized SQL schema for a Library System with Books, Members, and Loans. Suggest indexing for performance. Generate queries to find issued books, overdue books (more than 30 days), and total books borrowed by each member.

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

o Retrieve all orders by a user.

o Find the most purchased product.

o Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query optimization.

Expected Output:

• Complete schema with relationships + SQL queries for Analytics

```
CREATE TABLE Members (
    member_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);
CREATE TABLE Books (
    book_id INT PRIMARY KEY,
    title VARCHAR(200),
    author VARCHAR(100),
    available_copies INT
);
CREATE TABLE Loans (
    loan_id INT PRIMARY KEY,
    book_id INT,
    member_id INT,
    loan_date DATE,
```

```
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
CREATE INDEX idx_book_id ON Loans(book_id);
CREATE INDEX idx_member_id ON Loans(member_id);
INSERT INTO Members (member_id, name, email) VALUES
(1, 'Anil Kumar', 'anil@gmail.com'),
(2, 'Divya Sharma', 'divya@gmail.com'),
(3, 'Karthik R', 'karthik@gmail.com');
INSERT INTO Books (book_id, title, author, available_copies) VALUES
(101, 'Database Systems', 'Navathe', 5),
(102, 'Operating Systems', 'Galvin', 3),
(103, 'Python Programming', 'Guido', 4);
INSERT INTO Loans (loan_id, book_id, member_id, loan_date, return_date) VALUES
(1, 101, 1, '2026-02-10', NULL),
(2, 102, 2, '2026-01-15', NULL),
(3, 103, 3, '2026-03-01', '2026-03-10');
```

- Tables: Users, Products, Orders, OrderDetails.

```
SELECT u.name, o.order_id, o.order_date
FROM Users u
JOIN Orders o ON u.user_id = o.user_id
WHERE u.user_id = 1;
```

OrderDetails

order_detail_id	order_id	product_id	quantity
empty			

Orders

order_id	user_id	order_date
1	1	2026-03-10
2	2	2026-03-15

• Generate queries to:

o Retrieve all orders by a user.

o Find the most purchased product.

o Calculate total revenue in a given month.

```sql
SELECT p.product_name, SUM(od.quantity) AS total_sold
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.product_name
ORDER BY total_sold DESC
LIMIT 1;
```

Products

product_id	product_name	price
101	Laptop	60000
102	Headphones	2000
103	Mouse	500

• AI should also suggest normalization improvements and query optimization.

```sql
SELECT SUM(p.price * od.quantity) AS total_revenue
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
WHERE MONTH(o.order_date) = 3
AND YEAR(o.order_date) = 2026;
```

Users

user_id	name	email
1	Rahul	rahul@gmail.com
2	Sneha	sneha@gmail.com

PROMT:

Create an e-commerce database schema with Users, Products, Orders, and OrderDetails. Include relationships and normalization. Generate queries to get user orders, find the most purchased product, and calculate monthly revenue. Suggest query optimizations.

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

• Tables: Students, Subjects, Exams, Results.

• Define primary keys, foreign keys, and referential integrity constraints.

• Generate queries:

  o Insert examination results for a student.

  o Retrieve subject-wise marks for a student.

  o Calculate average marks for each subject.

Expected Output:

• Normalized schema and SQL queries involving aggregation and Joins

```sql
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50)
);
CREATE TABLE Subjects (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(100)
);
CREATE TABLE Exams (
    exam_id INT PRIMARY KEY,
    subject_id INT,
    exam_date DATE,
    FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
);
```

```sql
CREATE TABLE Results (
    result_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INT,
    exam_id INT,
    marks INT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
);


INSERT INTO Students (student_id, name, department) VALUES
(101, 'Asha', 'CSE'),
(102, 'Vikram', 'ECE');
```

Instructions:

• Tables: Students, Subjects, Exams, Results.

```
INSERT INTO Subjects (subject_id, subject_name) VALUES
(201, 'DBMS'),
(202, 'Data Structures');

INSERT INTO Exams (exam_id, subject_id, exam_date) VALUES
(301, 201, '2026-03-01'),
(302, 202, '2026-03-05');

INSERT INTO Results (student_id, exam_id, marks) VALUES
(101, 301, 85),
(101, 302, 78),
(102, 301, 88),
(102, 302, 90);
INSERT INTO Results (student_id, exam_id, marks)
VALUES (101, 301, 86);
```

• Define primary keys, foreign keys, and referential integrity constraints.

```
SELECT s.name, sub.subject_name, r.marks
FROM Results r
JOIN Students s ON r.student_id = s.student_id
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects sub ON e.subject_id = sub.subject_id
WHERE s.student_id = 101;
```

• Generate queries:

    o Insert examination results for a student.

o Retrieve subject-wise marks for a student.

o Calculate average marks for each subject.

```
SELECT sub.subject_name, AVG(r.marks) AS average_marks
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects sub ON e.subject_id = sub.subject_id
GROUP BY sub.subject_name;
```

OUTPUT:

Output

name	subject_name	marks
Asha	DBMS	85
Asha	Data Structures	78
Asha	DBMS	86

subject_name	average_marks
DBMS	86.33333333333333
Data Structures	84

PROMT:

Design a SQL schema for a University Examination System with Students, Subjects, Exams, and Results. Define primary and foreign keys. Generate queries to insert results, get subject-wise marks for a student, and calculate average marks per subject.